

Kirana Shop Management System - Database Documentation

Overview

This is a multi-tenant database system designed for local kirana (retail) shops. Multiple retailers share a single database, with data isolation achieved through `retailer_id` in each table.

Application Screens Summary

Screen 1: Purchase Management

Purpose: Record inventory purchases from suppliers

Key Operations: - Select supplier and add items to purchase order - Record quantities and purchase prices - Update inventory stock levels when order is received - Track payment status to suppliers

Tables Used: - `suppliers` - Select supplier - `items` - Search/add products - `purchase_orders` - Order header - `purchase_order_items` - Line items - `inventory_transactions` - Stock audit trail - `supplier_payments` - Payment tracking

Workflow: Supplier selection → Add items with qty/price → Create PO → Mark as received → Stock auto-updates → Record payments

Screen 2: Billing / Point of Sale

Purpose: Generate customer bills and process sales

Key Operations: - Search items by name/barcode - Verify stock availability - Add items to bill with quantities - Apply discounts and process payments - Auto-update inventory on sale

Tables Used: - `customers` - Optional customer lookup - `items` - Product search and pricing - `sales` - Bill header - `sale_items` - Line items with profit calculation - `inventory_transactions` - Stock reduction log

Workflow: Search items → Check stock → Add to bill → Apply discount → Select payment method → Generate bill → Stock auto-decreases

Screen 3: Stock Monitoring

Purpose: Monitor inventory levels and track stock movements

Key Operations: - View all items with current stock - Identify low stock items - Track complete stock movement history - Perform manual stock adjustments - Filter by category

Tables Used: - `items` - Inventory master with stock levels - `inventory_transactions` - Complete movement audit - `low_stock_items` (view) - Reorder alerts

Workflow: View inventory → Filter low stock → Check movement history → Manual adjustments if needed

Screen 4: Transaction History & Reports

Purpose: Review sales, purchases, payments and generate reports

Key Operations: - View sales history with filters - View purchase history - Track supplier payments and outstanding amounts - Generate daily/monthly summary reports - Analyze top-selling items

Tables Used: - `sales` + `sale_items` - Sales transactions - `purchase_orders` + `purchase_order_items` - Purchase history - `supplier_payments` - Payment records - `pending_supplier_payments` (view) - Outstanding dues - `daily_summary` - Pre-aggregated reports - `daily_sales_report` (view) - Sales breakdown

Workflow: Select date range → View transactions → Filter by supplier/customer → Track payments → Generate reports

Table Documentation

1. RETAILERS

Purpose: Master table storing information about each kirana shop using the system.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key, unique identifier	1, 2, 3
business_name	VARCHAR(255)	Shop name	"Sharma General Store", "Modi Kirana"
owner_name	VARCHAR(255)	Owner's full name	"Rajesh Sharma", "Amit Modi"
email	VARCHAR(255)	Contact email (unique across all retailers)	"rajesh@gmail.com"
phone_number	VARCHAR(15)	Contact number	"9876543210"
address	TEXT	Full shop address	"Shop 12, MG Road, Mumbai"
gst_number	VARCHAR(15)	GST registration number	"27AABCU9603R1ZM"
business_type	VARCHAR(50)	Type of business	"kirana", "general_store", "medical"
is_active	BOOLEAN	Whether shop is active	TRUE, FALSE
created_at	TIMESTAMP	When shop registered	"2024-01-15 10:30:00"
updated_at	TIMESTAMP	Last profile update	"2024-12-01 15:45:00"
deleted_at	TIMESTAMP	Soft delete timestamp (NULL if active)	NULL, "2024-12-15 12:00:00"

Usage: - Central reference for all retailer operations - Used for authentication and authorization - Filters all data queries by retailer_id

2. SUPPLIERS

Purpose: Stores information about suppliers/wholesalers from whom retailers purchase inventory.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer this supplier belongs to	5, 12, 23
company_name	VARCHAR(255)	Supplier company name	"ABC Distributors", "XYZ Wholesale"
contact_person	VARCHAR(255)	Person to contact	"Ramesh Kumar", "Suresh Patel"
phone	VARCHAR(15)	Contact number	"9123456789"
email	VARCHAR(255)	Email address	"abc@suppliers.com"
address	TEXT	Supplier address	"Warehouse 5, Industrial Area"
gst_number	VARCHAR(15)	Supplier's GST number	"24AABCU9603R1ZM"
created_at	TIMESTAMP	When supplier was added	"2024-02-10 09:00:00"
updated_at	TIMESTAMP	Last update	"2024-11-20 14:30:00"
deleted_at	TIMESTAMP	Soft delete	NULL

Usage: - **Screen 1:** Dropdown to select supplier when adding purchase order - **Screen 4:** View supplier payment history and outstanding amounts - Each retailer maintains their own supplier list

3. CUSTOMERS

Purpose: Stores customer information for each retailer (optional - for credit sales and customer history tracking).

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer this customer belongs to	5, 12
name	VARCHAR(255)	Customer name	"Priya Sharma", "Rahul Verma"
phone_number	VARCHAR(15)	Phone number	"9876543210"
email	VARCHAR(255)	Email (optional)	"priya@gmail.com", NULL
address	TEXT	Customer address	"Flat 201, Building A"
total_purchases	DECIMAL(12,2)	Lifetime purchase value	15000.50, 8500.00
created_at	TIMESTAMP	When customer was added	"2024-03-01 10:00:00"
updated_at	TIMESTAMP	Last update	"2024-12-15 16:20:00"
deleted_at	TIMESTAMP	Soft delete	NULL

Usage: - **Screen 2:** Search customer by phone for credit sales - Track repeat customers and purchase history - Generate customer-wise sales reports - Most walk-in sales can be without customer (customer_id = NULL)

4. ITEMS

Purpose: Product/inventory master table storing all items available in the shop.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer owns this item	5, 12
name	VARCHAR(255)	Product name	"Tata Salt 1kg", "Maggi Noodles"
description	TEXT	Product details	"Premium iodized salt", NULL
sku	VARCHAR(100)	Stock keeping unit (retailer's code)	"SALT-001", "MAG-200"
barcode	VARCHAR(100)	Product barcode	"8901030123450"
category	VARCHAR(100)	Product category	"Groceries", "Snacks", "Beverages"
unit	VARCHAR(50)	Unit of measurement	"kg", "liter", "piece", "packet"
purchase_price	DECIMAL(10,2)	Cost price from supplier	35.00, 120.50
selling_price	DECIMAL(10,2)	MRP to customer	40.00, 150.00
current_stock	DECIMAL(10,2)	Current available quantity	50.5, 100.0, 0.0
min_stock_threshold	DECIMAL(10,2)	Reorder level	10.0, 5.0
reorder_point	DECIMAL(10,2)	When to reorder	15.0, 8.0
max_stock_level	DECIMAL(10,2)	Maximum stock to maintain	200.0, 500.0
last_purchase_date	TIMESTAMP	Last time purchased from supplier	"2024-12-10 10:00:00"
last_sale_date	TIMESTAMP	Last time sold to customer	"2024-12-25 15:30:00"
is_active	BOOLEAN	Whether item is still sold	TRUE, FALSE
created_at	TIMESTAMP	When item was added	"2024-01-20 09:00:00"
updated_at	TIMESTAMP	Last update	"2024-12-26 10:15:00"
deleted_at	TIMESTAMP	Soft delete	NULL

Usage: - **Screen 1:** Add items to purchase order, update purchase_price - **Screen 2:** Search items for billing, check stock availability - **Screen 3:** Monitor stock levels, identify low stock items - Core table - drives entire inventory management

5. PURCHASE_ORDERS

Purpose: Header table for purchases made from suppliers.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer is purchasing	5, 12
supplier_id	BIGINT	From which supplier	3, 7, 15
order_date	TIMESTAMP	When order was placed/ received	"2024-12-20 11:00:00"
status	VARCHAR(20)	Order status	"pending", "received", "cancelled"
total_amount	DECIMAL(12,2)	Total order value	25000.00, 15500.50
payment_status	VARCHAR(20)	Payment completion status	"pending", "partial", "paid"
paid_amount	DECIMAL(12,2)	Amount paid so far	10000.00, 25000.00
notes	TEXT	Additional notes	"Delivery by truck", NULL
created_at	TIMESTAMP	Record creation time	"2024-12-20 11:00:00"
updated_at	TIMESTAMP	Last update	"2024-12-22 14:30:00"
deleted_at	TIMESTAMP	Soft delete	NULL

Usage: - **Screen 1:** Create new purchase order when buying from supplier - **Screen 4:** View all purchases, track pending payments - One purchase order can have multiple items (linked via purchase_order_items)

6. PURCHASE_ORDER_ITEMS

Purpose: Line items for each purchase order - what items were purchased and at what price.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
purchase_order_id	BIGINT	Which purchase order	15, 23
item_id	BIGINT	Which item was purchased	45, 67
quantity	DECIMAL(10,2)	How many units purchased	50.0, 100.5
unit_price	DECIMAL(10,2)	Price per unit from supplier	35.00, 120.00
amount	DECIMAL(12,2)	Total (quantity × unit_price)	1750.00, 12060.00
created_at	TIMESTAMP	When added	"2024-12-20 11:05:00"

Usage: - **Screen 1:** Add multiple items to a single purchase order - Stores historical pricing (important for profit calculation) - Example: Purchase Order #123 might have: 10 packets of Maggi @ ₹12 each, 5 kg Sugar @ ₹40 each

7. SUPPLIER_PAYMENTS

Purpose: Track payments made to suppliers against purchase orders.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer is paying	5, 12
supplier_id	BIGINT	To which supplier	3, 7
purchase_order_id	BIGINT	Against which PO (can be NULL for advance)	15, NULL
payment_date	TIMESTAMP	When payment was made	"2024-12-22 16:00:00"
amount	DECIMAL(12,2)	Amount paid	5000.00, 15000.00
payment_method	VARCHAR(20)	How paid	"cash", "cheque", "upi", "bank_transfer"
reference_number	VARCHAR(100)	Cheque/Transaction number	"CHQ123456", "UTR987654321"
notes	TEXT	Additional notes	"Partial payment", NULL
created_at	TIMESTAMP	Record creation	"2024-12-22 16:05:00"

Usage: - **Screen 4:** Record supplier payments - Track payment history per supplier - Calculate outstanding amounts - Example: Paid ₹10,000 to ABC Distributors via UPI on 22-Dec

8. SALES

Purpose: Header table for customer bills/invoices.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key (Bill Number)	1, 2, 3
retailer_id	BIGINT	Which retailer is selling	5, 12
customer_id	BIGINT	To which customer (NULL for walk-in)	25, NULL
sale_date	TIMESTAMP	When sale was made	"2024-12-26 10:30:00"
total_amount	DECIMAL(12,2)	Sum of all items	850.00, 1250.50
discount	DECIMAL(10,2)	Discount given	50.00, 0.00
final_amount	DECIMAL(12,2)	Amount to be paid (total - discount)	800.00, 1250.50
payment_method	VARCHAR(20)	How customer paid	"cash", "card", "upi", "credit"
payment_status	VARCHAR(20)	Payment status	"paid", "pending", "partial"
paid_amount	DECIMAL(12,2)	Amount received	800.00, 500.00
notes	TEXT	Bill notes	"Customer will pay tomorrow", NULL
created_at	TIMESTAMP	Bill creation time	"2024-12-26 10:30:00"
updated_at	TIMESTAMP	Last update	"2024-12-26 10:35:00"
deleted_at	TIMESTAMP	Soft delete	NULL

Usage: - **Screen 2:** Create customer bills - **Screen 4:** View all sales transactions - One sale can have multiple items (linked via sale_items) - Example: Bill #456 has total ₹850, customer got ₹50 discount, paid ₹800 via UPI

9. SALE_ITEMS

Purpose: Line items for each sale - what items were sold and at what price.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
sale_id	BIGINT	Which bill/sale	456, 789
item_id	BIGINT	Which item was sold	45, 67
quantity	DECIMAL(10,2)	How many units sold	2.0, 5.5
unit_selling_price	DECIMAL(10,2)	Price per unit at time of sale	40.00, 150.00
unit_cost_price	DECIMAL(10,2)	Cost price at time of sale	35.00, 120.00
amount	DECIMAL(12,2)	Total (quantity × unit_selling_price)	80.00, 825.00
profit	DECIMAL(12,2)	Profit (quantity × (selling - cost))	10.00, 165.00
created_at	TIMESTAMP	When added to bill	"2024-12-26 10:32:00"

Usage: - **Screen 2:** Add items to customer bill - Stores snapshot of prices (important - prices may change later) - Calculates profit per item - Example: Sold 2 packets of Maggi @ ₹12 each = ₹24, profit = ₹4

Why snapshot prices? - Item prices in `items` table may change tomorrow - This table preserves what price was actually charged to customer - Ensures historical accuracy for reports and returns

10. INVENTORY_TRANSACTIONS

Purpose: Audit log of all stock movements - purchases, sales, adjustments.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer	5, 12
item_id	BIGINT	Which item's stock changed	45, 67
transaction_type	VARCHAR(20)	Type of movement	"purchase", "sale", "adjustment", "return"
quantity	DECIMAL(10,2)	Change in stock (+ or -)	+50.0, -2.0, +10.0
reference_type	VARCHAR(50)	What caused this change	"purchase_order", "sale", "manual_adjustment"
reference_id	BIGINT	ID of the causing record	15, 456
previous_stock	DECIMAL(10,2)	Stock before transaction	100.0, 48.0
new_stock	DECIMAL(10,2)	Stock after transaction	150.0, 46.0
notes	TEXT	Additional notes	"Stock received from supplier", "Manual correction"
created_by	VARCHAR(100)	Who made the change	"admin", "system"
created_at	TIMESTAMP	When changed	"2024-12-26 10:30:00"

Usage: - **Screen 3:** View complete stock movement history - Track why stock changed (purchase/sale/adjustment) - Audit trail for accountability - Debugging stock discrepancies - Example entries: - +100 kg Sugar (purchase from supplier) - -2 kg Sugar (sold to customer) - +5 kg Sugar (manual stock correction)

11. RETURNS

Purpose: Track product returns from customers.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer	5, 12
sale_id	BIGINT	Original sale (can be NULL)	456, NULL
customer_id	BIGINT	Which customer	25, NULL
return_date	TIMESTAMP	When returned	"2024-12-27 14:00:00"
total_amount	DECIMAL(12,2)	Total refund amount	150.00, 80.00
refund_status	VARCHAR(20)	Refund completion status	"pending", "refunded"
reason	TEXT	Why returned	"Expired product", "Wrong item"
created_at	TIMESTAMP	Record creation	"2024-12-27 14:05:00"

Usage: - Handle customer complaints - Track return patterns - Adjust inventory when products come back - Example: Customer returned expired Maggi packet, refunded ₹12

12. RETURN_ITEMS

Purpose: Line items for each return - what items were returned.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
return_id	BIGINT	Which return	5, 8
sale_item_id	BIGINT	Original sale item	123, 456
quantity	DECIMAL(10,2)	How many returned	1.0, 2.5
amount	DECIMAL(12,2)	Refund for this item	12.00, 100.00

Usage: - Track which specific items were returned - Link back to original sale for verification - Partial returns possible (return 1 out of 2 items bought)

13. DAILY_SUMMARY

Purpose: Pre-aggregated daily statistics for faster dashboard loading.

Field	Type	Description	Example Values
id	BIGSERIAL	Primary key	1, 2, 3
retailer_id	BIGINT	Which retailer	5, 12
summary_date	DATE	Which day	"2024-12-26"
total_sales	DECIMAL(12,2)	Total sales amount	45000.00
total_purchases	DECIMAL(12,2)	Total purchases amount	30000.00
total_profit	DECIMAL(12,2)	Total profit	15000.00
cash_sales	DECIMAL(12,2)	Cash payments received	20000.00
card_sales	DECIMAL(12,2)	Card payments received	10000.00
upi_sales	DECIMAL(12,2)	UPI payments received	15000.00
credit_sales	DECIMAL(12,2)	Credit sales (pending)	5000.00
total_transactions	INT	Number of bills	125
created_at	TIMESTAMP	When created	"2024-12-27 00:05:00"
updated_at	TIMESTAMP	Last update	"2024-12-27 23:59:59"

Usage: - **Dashboard:** Quick daily/monthly reports - Updated by background job or trigger - Avoids heavy aggregation queries - Example: On 26-Dec, shop made ₹45,000 in sales across 125 bills

Views Documentation

1. LOW_STOCK_ITEMS

Purpose: Quickly identify items that need reordering.

Columns: All item fields + business_name

Logic: Shows items where `current_stock <= min_stock_threshold`

Usage: - **Screen 3:** Alert retailer about low stock - Generate purchase orders proactively - Prevent stockouts - Example: Shows "Tata Salt (5 kg left, minimum 10 kg required)"

2. PENDING_SUPPLIER_PAYMENTS

Purpose: Track outstanding amounts owed to suppliers.

Columns: - Purchase order details - Supplier name - Total amount, paid amount, pending amount

Logic: Shows purchase orders where `payment_status != 'paid'`

Usage: - **Screen 4:** See which suppliers need payment - Cash flow management - Payment scheduling - Example: ABC Distributors - ₹15,000 pending from order dated 20-Dec

3. DAILY_SALES_REPORT

Purpose: Aggregated daily sales breakdown by payment method.

Columns: - retailer_id - sale_day (date) - total_bills (count) - total_sales, net_sales - cash_sales, card_sales, upi_sales

Logic: Groups sales by date and sums amounts by payment method

Usage: - **Dashboard:** Daily performance tracking - Compare sales across days - Payment method analysis - Example: On 26-Dec, sold ₹45,000 across 125 bills (₹20k cash, ₹15k UPI, ₹10k card)

Screen-wise Table Usage

Screen 1: Add Purchase from Supplier

Tables Used: 1. suppliers - Select supplier 2. items - Search and add items 3. purchase_orders - Create order header 4. purchase_order_items - Add line items 5. inventory_transactions - Log stock increase 6. items.current_stock - Update stock levels

Workflow: 1. Select supplier from dropdown 2. Add items with quantities and prices 3. Create purchase_order record 4. Insert purchase_order_items 5. Update items.current_stock (+quantity) 6. Insert inventory_transaction (type='purchase')

Screen 2: Create Customer Bill

Tables Used: 1. customers - Optional: search customer 2. items - Search items, check stock 3. sales - Create bill header 4. sale_items - Add line items 5. inventory_transactions - Log stock decrease 6. items.current_stock - Update stock levels

Workflow: 1. Search items by name/barcode 2. Check current_stock availability 3. Add items to bill with quantities 4. Calculate total, discount, final amount 5. Create sales record 6. Insert sale_items with profit calculation 7. Update items.current_stock (-quantity) 8. Insert inventory_transaction (type='sale')

Screen 3: Stock Monitoring

Tables/Views Used: 1. items - View all items with stock levels 2. low_stock_items - View items needing reorder 3. inventory_transactions - View movement history

Queries: - List all items with current stock - Filter by category - Sort by stock level - Show items below minimum threshold - View complete movement history for any item

Screen 4: Transaction History

Tables/Views Used: 1. `sales` - Customer sales history 2. `purchase_orders` - Supplier purchase history 3. `supplier_payments` - Payment tracking 4. `pending_supplier_payments` - Outstanding amounts 5. `daily_summary` - Aggregated reports

Features: - View all sales with dates, amounts, payment status - View all purchases with supplier details - Track supplier payments and dues - Filter by date range - Search by customer/supplier name

Key Design Principles

1. Multi-Tenancy

- Every table has `retailer_id` (except retailers table)
- All queries must filter by `retailer_id`
- Complete data isolation between retailers

2. Soft Deletes

- Most tables have `deleted_at` field
- Records are never actually deleted
- Allows data recovery and audit
- Queries use `WHERE deleted_at IS NULL`

3. Price Snapshots

- `sale_items` stores actual selling price and cost price
- `purchase_order_items` stores actual purchase price
- Prevents data loss when prices change in `items` table
- Ensures historical accuracy

4. Stock Tracking

- `items.current_stock` - current quantity
- `inventory_transactions` - complete audit log
- Every stock change creates a transaction record
- Can reconcile stock by summing transactions

5. Payment Tracking

- Supports partial payments
- Separate tables for supplier and customer payments
- Track payment method (cash/card/UPI)
- Monitor outstanding amounts

6. Performance Optimization

- Indexes on all foreign keys
- Composite indexes on (`retailer_id`, other_fields)

- `daily_summary` for pre-aggregated data
 - Views for common complex queries
-

Common Queries

Get Low Stock Items

```
SELECT * FROM low_stock_items
WHERE retailer_id = ?
ORDER BY current_stock ASC;
```

Today's Sales

```
SELECT COUNT(*) as bills, SUM(final_amount) as total
FROM sales
WHERE retailer_id = ?
  AND DATE(sale_date) = CURRENT_DATE
  AND deleted_at IS NULL;
```

Top Selling Items (Last 30 Days)

```
SELECT i.name, SUM(si.quantity) as total_sold
FROM sale_items si
JOIN items i ON si.item_id = i.id
JOIN sales s ON si.sale_id = s.id
WHERE s.retailer_id = ?
  AND s.sale_date >= NOW() - INTERVAL '30 days'
GROUP BY i.id, i.name
ORDER BY total_sold DESC
LIMIT 10;
```

Pending Supplier Payments

```
SELECT * FROM pending_supplier_payments
WHERE retailer_id = ?
ORDER BY order_date ASC;
```


Monthly Profit Report

```
SELECT
  DATE_TRUNC('month', sale_date) as month,
  SUM(final_amount) as revenue,
  SUM(si.profit) as profit
FROM sales s
JOIN sale_items si ON s.id = si.sale_id
WHERE s.retailer_id = ?
  AND s.deleted_at IS NULL
GROUP BY DATE_TRUNC('month', sale_date)
ORDER BY month DESC;
```

Data Integrity Rules

1. **Cannot delete items that have transactions** - Use `ON DELETE RESTRICT`
2. **Deleting a sale cascades to sale_items** - Use `ON DELETE CASCADE`
3. **Deleting a retailer deletes all their data** - Use `ON DELETE CASCADE`
4. **Stock must never be negative** - Add check constraint or application validation
5. **Payment amounts cannot exceed order amounts** - Application validation
6. **Unique constraint on (retailer_id, summary_date)** in daily_summary

Maintenance & Best Practices

Daily Tasks

- Run job to populate/update `daily_summary` table
- Backup database
- Clean up old soft-deleted records (after 90 days)

Weekly Tasks

- Analyze slow queries and add indexes
- Review low stock items
- Generate pending payment reports

Monthly Tasks

- Archive old transaction data
- Performance tuning
- Generate monthly financial reports

Security

- Never expose raw retailer_id to frontend
- Use session/token to identify current retailer
- All API queries must filter by authenticated retailer_id

- Validate stock availability before sales
- Prevent negative stock scenarios